

SME202_胡帅_12211735_Project

顶层模块图：

Top Module（顶层）					
Pll 功能：产生 148.5M 的时钟信号，用于系统的时钟同步和驱动其他模块。	HdmiWrapper 功能：HDMI 显示模块，将生成的 bitmap 同步到显示屏上，实现图像显示功能。	KeyDebounce 功能：按键消抖模块，用于处理按键的物理抖动，提供稳定的按键输入信号。	Wrapper 功能：游戏核心模块，负责游戏的主要逻辑处理，包括游戏的开关状态管理，运行速度控制，分数计算等。 （下含子模块）		UartWrapper 功能：串口发送模块，将 bitmap 数据通过串口发送到 PC，实现数据传输。
			Bitmap 功能：负责更新方块在 bitmap 上的状态，分为固定状态 (bitmap_h) 和移动状态 (bitmap_l)。同时反馈方块能否移动的信息。	Block Control 功能：负责控制方块的移动，接收按键信息和 bitmap 的反馈信息，实现方块的位移和交互。	

KeyDebounce：

输入：原始按键输入电平，keys；时钟信号，clk

输出：消抖后按键稳定电平， key_stable（经过 Top Module 转变为 pressed 的位移指令信号）

KeyDebounce→Top Module→ Wrapper(Block Control)

传递按键信息 (key) 变换的位移指令 (pressed)

Wrapper:

输入: 位移指令, pressed; 时钟信号, clk (; 来自子模块的, 消除的行数, score)

输出: 游戏状态, game state; 游戏速度, game speed; 游戏分数, game score

Top Module (KeyDebounce) →Wrapper

根据按键信息变更游戏状态 game state 与速度 game speed

Bitmap→Wrapper

传递消除的行数 score 帮助判断游戏分数的变化

Bitmap:

输入: 方块位置和内容的信息, cur blk; 时钟信号, clk

输出: 更新 bitmap; 方块位移的反馈, cur blk act; 消除的行数, score

Block Control:

输入: 方块位移的反馈, cur blk act; 位移指令, pressed; 时钟信号, clk

输出: 方块位置和内容的信息, cur blk;

Block Control→Bitmap

Bitmap/Top Module (KeyDebounce) →Block Control

Bitmap 接收方块的基础信息 (cur blk) 并作用在 bitmap 上 (传递到 hmi 中), 处理并反馈 cur blk act 给 Block Control; Block Control 创建方块基础信息 (cur blk) 并根据接收到的位移指令 (pressed) 在反馈 (cur blk act) 上变更方块状态。

SegWrapper:

输入: 游戏得分, game score; 时钟信号, clk

输出: 数码管控制片选信号, seg_sel; 单个数码管 LED 控制电平, seg_data

Wrapper→SegWrapper

接收 game score 处理到各个数码管 seg_sel 以不同的 seg_data

代码部分:

按键消除抖动:

```
module KeyDebounce #(
    // clock frequency(Mhz), 50 MHz
    parameter CLK_FREQ = 50_000_000,
    parameter KEY_CNT = 8
```

```

)
(
    input                clk,                // clock input
    input [KEY_CNT-1:0] keys,                // input key pins, raw
input
    output [KEY_CNT-1:0] keys_stable        // output stable key
status, 0 - press down
);

// TODO - add your logic
reg [KEY_CNT-1:0] keys_prev; // 用于缓存前一个时刻的按键状态
reg [KEY_CNT-1:0] keys_debounced=8'b1111_1111; // 用于存储消抖后的按键状态

reg [19:0] debounce_counter; // 消抖计数器, 20ms 为一个计数周期

// 按键状态的变化检测
always @(posedge clk) begin
    keys_prev <= keys; // 缓存当前的按键状态

    // 如果按键状态有变化, 则重新开始消抖计数
    if (keys != keys_prev)
        debounce_counter <= 0;
    else if (debounce_counter < (CLK_FREQ / 50)) // 20ms 的消抖
        debounce_counter <= debounce_counter + 1;
end

// 按键消抖
always @(posedge clk) begin
    if (debounce_counter == (CLK_FREQ / 50)) begin
        keys_debounced <= keys; // 当消抖计数器达到阈值时, 更新消抖
后的按键状态
    end
end

// 输出稳定的按键状态
assign keys_stable[KEY_CNT-1:0] = keys_debounced[KEY_CNT-1:0];

endmodule

```

方块位移 blk_control:

//基础信息输入

```

//输入 bitmap 中的反馈变量
input cur_blk_act, // 判断能否下移, 高电平有效
input cur_blk_act_rot, // 判断能否旋转, 高电平有效
input cur_blk_act_lef, // 判断能否左移, 高电平有效
input cur_blk_act_rig // 判断能否右移, 高电平有效

```

```

//缓存行列以及方块信息的变量
reg [ROW_ADDR_W-1:0] cur_blk_row_r;
reg [COL_ADDR_W-1:0] cur_blk_col_r;
reg [15:0] cur_blk_data_r;
assign cur_blk_row = cur_blk_row_r;
assign cur_blk_col = cur_blk_col_r;
assign cur_blk_data = cur_blk_data_r;

```

```

task reset_cur_blk(); //方块归零更新重开
begin
    cur_blk_row_r <= 0;
    cur_blk_col_r <= (AREA_COL >> 1) - 2;
    cur_blk_data_r <= nxt_blk_data;
end
endtask

```

//方块上下左右移动

```

always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        reset_cur_blk();
    end
    else if (~cur_blk_act) begin //不能下移则要重新放方块
        reset_cur_blk();
    end
    // 1) when current block falling down
    else if (falling_update) begin
        // TODO - add your logic
        if (cur_blk_act) begin //如果方块能够下移
            cur_blk_row_r <= cur_blk_row_r + 1; //行数+1
        end
    end
    // 2) when has pressed keys - UP (rotate)
    else if (pressed_up) begin
        // TODO - add your logic
        if (cur_blk_act_rot) begin //如果方块能够旋转, 旋转 90°
            cur_blk_data_r <= {cur_blk_data_r[12], cur_blk_data_r[8],
cur_blk_data_r[4], cur_blk_data_r[0],

```

```

        cur_blk_data_r[13], cur_blk_data_r[9],
cur_blk_data_r[5], cur_blk_data_r[1],
        cur_blk_data_r[14], cur_blk_data_r[10],
cur_blk_data_r[6], cur_blk_data_r[2],
        cur_blk_data_r[15], cur_blk_data_r[11],
cur_blk_data_r[7], cur_blk_data_r[3]}};
    end
end
// 3) when has pressed keys - DOWN
else if (pressed_down) begin
    // TODO - add your logic
    if (cur_blk_act) begin//如果方块能够下移
        cur_blk_row_r <= cur_blk_row_r + 1;//行数+1
    end
end
// 4) when has pressed keys - LEFT
else if (pressed_left) begin
    // TODO - add your logic
    if (cur_blk_act_lef) begin//如果方块能够左移
        if (cur_blk_col_r == 0) begin //如果左侧边角无方块，col 无法，继续-1，因此需要额外的 if 进行判断，此时方块的信息整体左移，又考虑到最左边的一列信息无效（全为0），因此处理上最右边的一列设定为0，其余信息向左移动，起到同样的移动效果
            cur_blk_data_r[0] <= cur_blk_data_r[1];
            cur_blk_data_r[1] <= cur_blk_data_r[2];
            cur_blk_data_r[2] <= cur_blk_data_r[3];
            cur_blk_data_r[3] <= 0;
            cur_blk_data_r[4] <= cur_blk_data_r[5];
            cur_blk_data_r[5] <= cur_blk_data_r[6];
            cur_blk_data_r[6] <= cur_blk_data_r[7];
            cur_blk_data_r[7] <= 0;
            cur_blk_data_r[8] <= cur_blk_data_r[9];
            cur_blk_data_r[9] <= cur_blk_data_r[10];
            cur_blk_data_r[10] <= cur_blk_data_r[11];
            cur_blk_data_r[11] <= 0;
            cur_blk_data_r[12] <= cur_blk_data_r[13];
            cur_blk_data_r[13] <= cur_blk_data_r[14];
            cur_blk_data_r[14] <= cur_blk_data_r[15];
            cur_blk_data_r[15] <= 0;
        end
    end
    else begin
        cur_blk_col_r <= cur_blk_col_r - 1;//正常情况下 col 不为
0 时左移
    end
end

```

```

        end
    end
    // 5) when has pressed keys - RIGHT
    else if (pressed_right) begin
        // TODO - add your logic
        if(cur_blk_act_rig)begin//如果方块能够右移
            if(cur_blk_col+3 == AREA_COL-1) begin//如果右侧边角无方块，
col 无法，继续+1，因此需要额外的 if 进行判断，此时方块的信息整体右移，
又考虑到最右边的一列信息无效（全为 0），因此处理上最左边的一列设定为
0，其余信息向右移动，起到同样的移动效果
                cur_blk_data_r[0]  <= 0;
                cur_blk_data_r[1]  <= cur_blk_data_r[0];
                cur_blk_data_r[2]  <= cur_blk_data_r[1];
                cur_blk_data_r[3]  <= cur_blk_data_r[2];
                cur_blk_data_r[4]  <= 0;
                cur_blk_data_r[5]  <= cur_blk_data_r[4];
                cur_blk_data_r[6]  <= cur_blk_data_r[5];
                cur_blk_data_r[7]  <= cur_blk_data_r[6];
                cur_blk_data_r[8]  <= 0;
                cur_blk_data_r[9]  <= cur_blk_data_r[8];
                cur_blk_data_r[10] <= cur_blk_data_r[9];
                cur_blk_data_r[11] <= cur_blk_data_r[10];
                cur_blk_data_r[12] <= 0;
                cur_blk_data_r[13] <= cur_blk_data_r[12];
                cur_blk_data_r[14] <= cur_blk_data_r[13];
                cur_blk_data_r[15] <= cur_blk_data_r[14];
            end
        else begin
            cur_blk_col_r <= cur_blk_col_r+1; //正常情况下 col 不为最
右-3 时右移
        end
    end
end
end
end

```

bitmap 模块:

//触底或触壁逻辑

```

reg [31:0] touch; //设置缓存变量检测每行是否能够发生碰撞
generate
    genvar i;
    for(i = 0; i < AREA_ROW; i = i + 1) begin //重复检查每一行的状态
        if ( i == 31 )

```

```

        always @(*) begin
            if(bitmap_l[31] != 1'b0) begin //触底判断，当 bitmap_l 最后一行任意位置存在方块即说明当前方块到底底部
                touch[31] = 1'b1; //设定 touch 最后一行发生了碰撞
            end
            else begin
                touch[31] = 1'b0; //否则没有触底
            end
        end
    end
else begin
    always @(*)begin
        if((bitmap_l[i] & bitmap_h[i + 1]) != 1'b0) begin //触壁判断，检查 bitmap 当前位置的方块能否被放入下一行同一列，若都能被放入即说明没有发生碰壁，否则任意位置不能被放入即视为碰壁
            touch[i] = 1'b1; //设定该行碰壁
        end
        else begin
            touch[i] = 1'b0; //否则没有发生碰壁
        end
    end
end
end
endgenerate

```

assign cur_blk_act = ~(|touch); //若没有任何地方发生碰撞（均为 0），则方块可以下移（1）；否则只要有一行发生碰壁或触底就不能下移（0）

//方块确认，行数消除以及计数反馈

```

output [2:0] score, //消除行数计数反馈到 wrapper
//消除行数计算器
wire [31:0] elirow; //行数消除记录，高电平表示发生了消除
reg [31:0] elirow_r;
assign elirow=elirow_r;
assign score =
    elirow[0]+elirow[1]+elirow[2]+elirow[3]+elirow[4]+elirow[5]+elirow[6]
    +elirow[7]+elirow[8]+elirow[9]+

    elirow[10]+elirow[11]+elirow[12]+elirow[13]+elirow[14]+elirow[15]+eli
    row[16]+elirow[17]+elirow[18]+elirow[19]+

    elirow[20]+elirow[21]+elirow[22]+elirow[23]+elirow[24]+elirow[25]+eli
    row[26]+elirow[27]+elirow[28]+elirow[29]+
    elirow[30]+elirow[31];

```

//score 反馈变量，表示所有消除的行数

```
generate
integer o;
integer p;
integer m;
integer n;
always @(posedge clk or negedge rstn) begin
    if (~rstn) begin //归零时，清空 bitmap_h
        for (m = 0; m < AREA_ROW; m = m + 1) begin
            bitmap_h[m] <= 16'h0;
        end
    end
    else begin
        if (~cur_blk_act) begin //触底或触壁
            for (m = 0; m < AREA_ROW; m = m + 1) begin
                bitmap_h[m] <= bitmap_h[m] | bitmap_l[m]; //浮动方块被确
                认到 bitmap_h 中
            end
        end
        for (p = 0; p < AREA_ROW; p = p + 1) begin//从上往下进行消行判
            断
                elirow_r[p] <= (bitmap_h[p] == 16'hFFFF);//当发生满行时设定
                消行计数器对应位置变为高电平
            end
            for (o = 0; o < AREA_ROW; o = o + 1) begin
                if(bitmap_h[o] == 16'hFFFF)begin //从上往下进行消行判断，确
                认满行的位置
                    if(o > 0) begin //满行不在第一行时
                        for (n = o; n > 0; n = n - 1)begin
                            bitmap_h[n] <= (bitmap_h[n - 1]); //满行之上每一行
                            下移一格
                        end
                        bitmap_h[0] <= 16'h0; //首行清空
                    end
                    else begin
                        bitmap_h[0] <= 16'h0; //首行清空
                    end
                end
            end
        end
    end
end
endgenerate
```


//左右移动以及旋转判断（方法类似）

//判断是否可以左移

```
reg cur_blk_act_lef_r;
assign cur_blk_act_lef = cur_blk_act_lef_r;

//对每一个方块位置进行对比，判断是否会发生堵塞
wire [15:0] lef_result; //记录每个方块能否变动的结果，高电平表示不能移动
wire lef_tot_result; //综合结果，只要有一个被堵塞，则说明不能左移，高电平表示不能移动
assign lef_result[0] = (bitmap_h[cur_blk_row][cur_blk_col-1] &
bitmap_l[cur_blk_row][cur_blk_col]);
assign lef_result[1] = (bitmap_h[cur_blk_row+1][cur_blk_col-1] &
bitmap_l[cur_blk_row+1][cur_blk_col]);
assign lef_result[2] = (bitmap_h[cur_blk_row+2][cur_blk_col-1] &
bitmap_l[cur_blk_row+2][cur_blk_col]);
assign lef_result[3] = (bitmap_h[cur_blk_row+3][cur_blk_col-1] &
bitmap_l[cur_blk_row+3][cur_blk_col]);
assign lef_result[4] = (bitmap_h[cur_blk_row][cur_blk_col-1+1] &
bitmap_l[cur_blk_row][cur_blk_col+1]);
assign lef_result[5] = (bitmap_h[cur_blk_row+1][cur_blk_col-1+1] &
bitmap_l[cur_blk_row+1][cur_blk_col+1]);
assign lef_result[6] = (bitmap_h[cur_blk_row+2][cur_blk_col-1+1] &
bitmap_l[cur_blk_row+2][cur_blk_col+1]);
assign lef_result[7] = (bitmap_h[cur_blk_row+3][cur_blk_col-1+1] &
bitmap_l[cur_blk_row+3][cur_blk_col+1]);
assign lef_result[8] = (bitmap_h[cur_blk_row][cur_blk_col-1+2] &
bitmap_l[cur_blk_row][cur_blk_col+2]);
assign lef_result[9] = (bitmap_h[cur_blk_row+1][cur_blk_col-1+2] &
bitmap_l[cur_blk_row+1][cur_blk_col+2]);
assign lef_result[10] = (bitmap_h[cur_blk_row+2][cur_blk_col-1+2] &
bitmap_l[cur_blk_row+2][cur_blk_col+2]);
assign lef_result[11] = (bitmap_h[cur_blk_row+3][cur_blk_col-1+2] &
bitmap_l[cur_blk_row+3][cur_blk_col+2]);
assign lef_result[12] = (bitmap_h[cur_blk_row][cur_blk_col-1+3] &
bitmap_l[cur_blk_row][cur_blk_col+3]);
assign lef_result[13] = (bitmap_h[cur_blk_row+1][cur_blk_col-1+3] &
bitmap_l[cur_blk_row+1][cur_blk_col+3]);
assign lef_result[14] = (bitmap_h[cur_blk_row+2][cur_blk_col-1+3] &
bitmap_l[cur_blk_row+2][cur_blk_col+3]);
assign lef_result[15] = (bitmap_h[cur_blk_row+3][cur_blk_col-1+3] &
bitmap_l[cur_blk_row+3][cur_blk_col+3]);
```

```

assign lef_tot_result = |lef_result;
always @(*) begin
    if(cur_blk_col==0 && (cur_blk_data[0] | cur_blk_data[4] |
cur_blk_data[8] | cur_blk_data[12]))begin //判断在最左边靠墙时最左侧
是否存在方块阻挡左移
        cur_blk_act_lef_r = 0; //被阻挡，不能动
    end
    else if(lef_tot_result)begin //判断其余情况下能否移动
        cur_blk_act_lef_r = 0; //被阻挡，不能动
    end
    else begin
        cur_blk_act_lef_r = 1; //可以移动
    end
end
end

```

//判断是否可以右移

```

reg cur_blk_act_rig_r;
assign cur_blk_act_rig = cur_blk_act_rig_r;

//对每一个方块位置进行对比，判断是否会发生堵塞
wire [15:0] rig_result; //记录每个方块能否变动的结果，，高电平表示不
能移动
wire rig_tot_result; //综合结果，只要有一个被堵塞，则说明不能右移，高
电平表示不能移动
assign rig_result[0] = (bitmap_h[cur_blk_row][cur_blk_col+1] &
bitmap_l[cur_blk_row][cur_blk_col]);
assign rig_result[1] = (bitmap_h[cur_blk_row+1][cur_blk_col+1] &
bitmap_l[cur_blk_row+1][cur_blk_col]);
assign rig_result[2] = (bitmap_h[cur_blk_row+2][cur_blk_col+1] &
bitmap_l[cur_blk_row+2][cur_blk_col]);
assign rig_result[3] = (bitmap_h[cur_blk_row+3][cur_blk_col+1] &
bitmap_l[cur_blk_row+3][cur_blk_col]);
assign rig_result[4] = (bitmap_h[cur_blk_row][cur_blk_col+1+1] &
bitmap_l[cur_blk_row][cur_blk_col+1]);
assign rig_result[5] = (bitmap_h[cur_blk_row+1][cur_blk_col+1+1] &
bitmap_l[cur_blk_row+1][cur_blk_col+1]);
assign rig_result[6] = (bitmap_h[cur_blk_row+2][cur_blk_col+1+1] &
bitmap_l[cur_blk_row+2][cur_blk_col+1]);
assign rig_result[7] = (bitmap_h[cur_blk_row+3][cur_blk_col+1+1] &
bitmap_l[cur_blk_row+3][cur_blk_col+1]);
assign rig_result[8] = (bitmap_h[cur_blk_row][cur_blk_col+1+2] &
bitmap_l[cur_blk_row][cur_blk_col+2]);

```

```

assign rig_result[9] = (bitmap_h[cur_blk_row+1][cur_blk_col+1+2] &
bitmap_l[cur_blk_row+1][cur_blk_col+2]);
assign rig_result[10] = (bitmap_h[cur_blk_row+2][cur_blk_col+1+2] &
bitmap_l[cur_blk_row+2][cur_blk_col+2]);
assign rig_result[11] = (bitmap_h[cur_blk_row+3][cur_blk_col+1+2] &
bitmap_l[cur_blk_row+3][cur_blk_col+2]);
assign rig_result[12] = (bitmap_h[cur_blk_row][cur_blk_col+1+3] &
bitmap_l[cur_blk_row][cur_blk_col+3]);
assign rig_result[13] = (bitmap_h[cur_blk_row+1][cur_blk_col+1+3] &
bitmap_l[cur_blk_row+1][cur_blk_col+3]);
assign rig_result[14] = (bitmap_h[cur_blk_row+2][cur_blk_col+1+3] &
bitmap_l[cur_blk_row+2][cur_blk_col+3]);
assign rig_result[15] = (bitmap_h[cur_blk_row+3][cur_blk_col+1+3] &
bitmap_l[cur_blk_row+3][cur_blk_col+3]);
assign rig_tot_result = |rig_result;
always @(*) begin
    if(cur_blk_col+3==AREA_COL-1 && (cur_blk_data[3] |
cur_blk_data[7] | cur_blk_data[11] | cur_blk_data[15]))begin//判断在
最右边靠墙时最右侧是否存在方块阻挡右移
        cur_blk_act_rig_r = 0; //被阻挡，不能动
    end
    else if(rig_tot_result)begin //判断其余情况下能否移动
        cur_blk_act_rig_r = 0; //被阻挡，不能动
    end
    else begin
        cur_blk_act_rig_r = 1; //可以移动
    end
end
end

```

//判断是否可以旋转

```

reg cur_blk_act_rot_r;
assign cur_blk_act_rot = cur_blk_act_rot_r;

```

```

//对每一个方块位置进行对比，判断是否会发生堵塞
wire [15:0] rot_result; //记录每个方块能否变动的结果，，高电平表示不
能移动
wire rot_tot_result; //综合结果，只要有一个被堵塞，则说明不能右移，高
电平表示不能移动
assign rot_result[0] = cur_blk_data[12] &
bitmap_h[cur_blk_row][cur_blk_col];
assign rot_result[1] = cur_blk_data[8] &
bitmap_h[cur_blk_row][cur_blk_col+1];

```

```

assign rot_result[2] = cur_blk_data[4] &
bitmap_h[cur_blk_row][cur_blk_col+2];
assign rot_result[3] = cur_blk_data[0] &
bitmap_h[cur_blk_row][cur_blk_col+3];
assign rot_result[4] = cur_blk_data[13] &
bitmap_h[cur_blk_row+1][cur_blk_col];
assign rot_result[5] = cur_blk_data[9] &
bitmap_h[cur_blk_row+1][cur_blk_col+1];
assign rot_result[6] = cur_blk_data[5] &
bitmap_h[cur_blk_row+1][cur_blk_col+2];
assign rot_result[7] = cur_blk_data[1] &
bitmap_h[cur_blk_row+1][cur_blk_col+3];
assign rot_result[8] = cur_blk_data[14] &
bitmap_h[cur_blk_row+2][cur_blk_col];
assign rot_result[9] = cur_blk_data[10] &
bitmap_h[cur_blk_row+2][cur_blk_col+1];
assign rot_result[10] = cur_blk_data[6] &
bitmap_h[cur_blk_row+2][cur_blk_col+2];
assign rot_result[11] = cur_blk_data[2] &
bitmap_h[cur_blk_row+2][cur_blk_col+3];
assign rot_result[12] = cur_blk_data[15] &
bitmap_h[cur_blk_row+3][cur_blk_col];
assign rot_result[13] = cur_blk_data[11] &
bitmap_h[cur_blk_row+3][cur_blk_col+1];
assign rot_result[14] = cur_blk_data[7] &
bitmap_h[cur_blk_row+3][cur_blk_col+2];
assign rot_result[15] = cur_blk_data[3] &
bitmap_h[cur_blk_row+3][cur_blk_col+3];
assign rot_tot_result = |rot_result;
always @(*) begin
    if(rot_tot_result) begin //当任意一处发生阻挡
        cur_blk_act_rot_r = 0; //无法移动
    end
    else begin
        cur_blk_act_rot_r = 1; //否则可以移动
    end
end
end

```

游戏状态和速度控制:

```

//=====
=====

```

// game state and speed

//=====

reg game_state = 1'b1; // 1 - started, 0 - puased
reg game_speed = 1'b0; // 0 - normal spend; 1 - 2x speed

reg [25:0] falling_clk_cnt = 26'b0;

assign falling_update = (falling_clk_cnt >= (SPEED_FREQ - 1)) ?
game_state : 1'b0;

```
always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        falling_clk_cnt <= 26'b0; //复位
    end
    else if (falling_clk_cnt >= (SPEED_FREQ - 1)) begin
        falling_clk_cnt <= 26'b0; //打满归零
    end
    else if (game_speed) begin
        falling_clk_cnt <= falling_clk_cnt + 26'd2; //2 倍速
    end
    else begin
        falling_clk_cnt <= falling_clk_cnt + 26'b1; //1 倍速
    end
end
```

```
always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        game_state <= 1'b1; //复位
    end
    else if (pressed_pause_or_start) begin
        game_state <= ~game_state; //开关切换
    end
end
```

```
always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        game_speed <= 1'b0; //复位
    end
    else if (pressed_speed_up_down) begin
        game_speed <= ~game_speed; //速度切换
    end
end
```

```

//game over

wire response;
assign game_over = response;

//=====

// game score

//=====

reg [9:0] game_score_r = 0;
assign game_score = game_score_r;
wire [2:0] score; //bitmap 中输出的变量，记录了一次行动下总共消除的行
数
// TODO - add your logic

always @(posedge clk or negedge rstn) begin
    if(~rstn) begin
        game_score_r <= 0;
    end
    else begin
        case(score)
            3'b001: game_score_r <= game_score_r + 1;//消除的行数为
1, 加 1 分
            3'b010: game_score_r <= game_score_r + 1;//消除的行数为
2, 在加 1 分的基础上再加 1 分，共 2 分
            3'b011: game_score_r <= game_score_r + 2;//消除的行数为
3, 在加 2 分的基础上再加 2 分，共 4 分
            3'b100: game_score_r <= game_score_r + 4;//消除的行数为
4, 在加 4 分的基础上再加 4 分，共 8 分
            default: game_score_r <= game_score_r; //分数在其他反馈时
不变
        endcase
    end
end
end

```

分数输出

```

`define SEG_SEL_NULL 4'b0000
`define SEG_SEL_0 4'b0001
`define SEG_SEL_1 4'b0010

```

```

`define SEG_SEL_2 4'b0100
`define SEG_SEL_3 4'b1000
//自定义各 segment 的表示参数

// `define SEG_FLASH_DUR 26'd49_999

module SegWrapper #(
    parameter CLK_FREQ = 50_000_000,
    parameter SEG_FLASH_DUR = 49_999,
    parameter SCAN_FREQ = 200, // scan frequency
    parameter SCAN_CLK_CNT = CLK_FREQ / (SCAN_FREQ * 4) - 1
) (
    input clk,
    input rstn,
    input [9:0] game_score,
    output reg [3:0] seg_sel,
    output reg [7:0] seg_data
);

// 计数器, 触发 scan_next 信号
reg [31:0] clk_cnt = 32'b0;
wire scan_next;
assign scan_next = (clk_cnt == SCAN_CLK_CNT);

always @(posedge clk or negedge rstn) begin
    if (rstn == 1'b0) begin
        clk_cnt <= 32'b0;
    end else begin
        if (clk_cnt == SCAN_CLK_CNT) begin
            clk_cnt <= 32'b0;
        end else begin
            clk_cnt <= clk_cnt + 32'b1;
        end
    end
end

// 状态机 for seg_sel
reg [3:0] next_seg_sel = `SEG_SEL_NULL;

// 状态机 for seg_sel, 1) 状态转移, 时序逻辑
always @(posedge clk or negedge rstn) begin
    if (rstn == 1'b0) begin
        seg_sel <= `SEG_SEL_NULL;
    end else begin

```

```

        seg_sel <= next_seg_sel; //正常状态下不断变化，更新不同的 segment
    end
end

// 状态机 for seg_sel, 2) 状态切换，组合逻辑
always @(*) begin
    if (scan_next) begin //当下一个更新时间到来
        case (seg_sel) //更新下一个的目标
            `SEG_SEL_NULL: next_seg_sel = `SEG_SEL_0;
            `SEG_SEL_0: next_seg_sel = `SEG_SEL_1;
            `SEG_SEL_1: next_seg_sel = `SEG_SEL_2;
            `SEG_SEL_2: next_seg_sel = `SEG_SEL_3;
            `SEG_SEL_3: next_seg_sel = `SEG_SEL_0;
            default: next_seg_sel = `SEG_SEL_NULL;
        endcase
    end else begin
        next_seg_sel = seg_sel; //保持目标
    end
end

// 状态机 for seg_sel, 3) 输出 sel_num & seg_data, 时序逻辑
reg [3:0] sel_num = 4'b0;

always @(posedge clk or negedge rstn) begin
    if (rstn == 1'b0) begin
        sel_num <= 4'b0;
    end else begin
        case (seg_sel) //更新目标对应的分数
            `SEG_SEL_NULL: sel_num <= 4'b0;
            `SEG_SEL_0: sel_num <= ((game_score / 1000) % 10); //分数的个位数
            `SEG_SEL_1: sel_num <= ((game_score / 100) % 10); //分数的十位数
            `SEG_SEL_2: sel_num <= ((game_score / 10) % 10); //分数的百位数
            `SEG_SEL_3: sel_num <= (game_score % 10); //分数的千位数
            default: sel_num <= 4'b0;
        endcase
    end
end

// combination logic, output: seg_data, based on sel_num
always @(*) begin
    case(sel_num)
        4'd0: seg_data = 8'b1100_0000;
        4'd1: seg_data = 8'b1111_1001;
        4'd2: seg_data = 8'b1010_0100;
    end
end

```



```

        4'd3: seg_data = 8'b1011_0000;
        4'd4: seg_data = 8'b1001_1001;
        4'd5: seg_data = 8'b1001_0010;
        4'd6: seg_data = 8'b1000_0010;
        4'd7: seg_data = 8'b1111_1000;
        4'd8: seg_data = 8'b1000_0000;
        4'd9: seg_data = 8'b1001_0000;
        4'ha: seg_data = 8'b1000_1000;
        4'hb: seg_data = 8'b1000_0011;
        4'hc: seg_data = 8'b1100_0110;
        4'hd: seg_data = 8'b1010_0001;
        4'he: seg_data = 8'b1000_0110;
        4'hf: seg_data = 8'b1000_1110;
        default: seg_data = 8'b1100_0000;
    endcase
end

endmodule

```

测试部分（含图像）：

Bitmap:

```

`timescale 1ns / 1ps

module tb_bitmap ();

    parameter AREA_ROW = 32;
    parameter AREA_COL = 16;
    parameter ROW_ADDR_W = 5;
    parameter COL_ADDR_W = 4;
    parameter SPEED_FREQ = 10;

    reg clk = 1'b0;
    reg rstn = 1'b1;

    wire falling_update;

    reg [ROW_ADDR_W-1:0] cur_blk_row = 0; // current moving block
    reg [COL_ADDR_W-1:0] cur_blk_col = 6; // : top-left
    position

```

```

wire cur_blk_act;
reg [15:0] cur_blk_data = {
    4'b0_0_0_0,
    4'b0_1_1_0,
    4'b0_0_1_0,
    4'b0_0_1_0
};

```

```

Bitmap #(
    .AREA_ROW (AREA_ROW),
    .AREA_COL (AREA_COL),
    .ROW_ADDR_W (ROW_ADDR_W),
    .COL_ADDR_W (COL_ADDR_W),
    .SPEED_FREQ (SPEED_FREQ)
) u_bitmap (
    .clk(clk),
    .rstn(rstn),
    .falling_update(falling_update),
    .cur_blk_row(cur_blk_row),
    .cur_blk_col(cur_blk_col),
    .cur_blk_data(cur_blk_data),
    .test_cur_blk_row(test_cur_blk_row),
    .test_cur_blk_col(test_cur_blk_col),
    .test_cur_blk_data(test_cur_blk_data),
    .cur_blk_act(cur_blk_act),
    .cur_blk_act_rot(cur_blk_act_rot),
    .cur_blk_act_lef(cur_blk_act_lef),
    .cur_blk_act_rig(cur_blk_act_rig),
    .score(score),
    .response(response),
    .r1_row(r1_row),
    .r1_data(r1_data),
    .r2_row(r2_row),
    .r2_data(r2_data)
);

```

```

initial begin

```

```

    rstn = 1;
    #10;
    rstn = 0;
    #10;
    rstn = 1;
    #10;

```

```

        #20000;
        $finish;
    end

    always #1 clk = ~clk;

    reg [25:0] falling_clk_cnt = 26'b0;

    assign falling_update = (falling_clk_cnt == (SPEED_FREQ - 1)) ? 1'b1 : 1'b0;

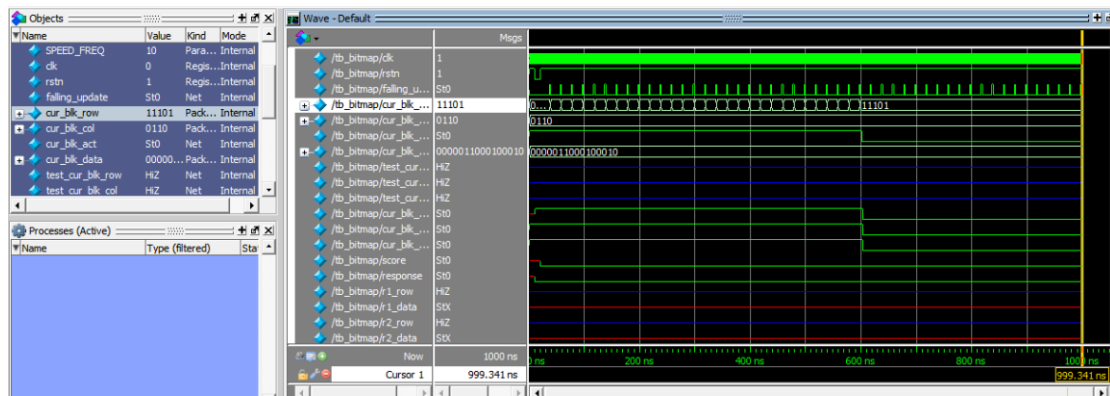
    always @(posedge clk or negedge rstn) begin
        if (~rstn) begin
            falling_clk_cnt <= 26'b0;
        end
        else if (falling_clk_cnt >= (SPEED_FREQ - 1)) begin
            falling_clk_cnt <= 26'b0;
        end
        else begin
            falling_clk_cnt <= falling_clk_cnt + 26'b1;
        end
    end

    always @(posedge clk or negedge rstn) begin
        if (~rstn) begin
            cur_blk_row <= 0;
        end
        else if (falling_update) begin
            if (cur_blk_act) begin
                cur_blk_row <= cur_blk_row + 1;
            end
            else begin
                cur_blk_row <= cur_blk_row;
            end
        end
    end

    // 输出波形文件
    initial begin
        $dumpfile("tb_bitmap.wave");
        $dumpvars;
        $display("save to tb_bitmap.wave");
    end
end

```

endmodule



测试结果显示该方块能够正常运行到底部静止，对应行列数据正确。

Wrapper:

```
`timescale 1ns / 1ps
```

```
module tb_Wrapper ();
```

```
parameter AREA_ROW = 32;
parameter AREA_COL = 16;
parameter ROW_ADDR_W = 5;
parameter COL_ADDR_W = 4;
parameter SPEED_FREQ = 50_000_000;
```

```
// Inputs
reg clk;
reg rstn;
reg pressed_left;
reg pressed_right;
reg pressed_up;
reg pressed_down;
reg pressed_speed_up_down;
reg pressed_pause_or_start;
reg [ROW_ADDR_W-1:0] r1_row;
reg [ROW_ADDR_W-1:0] r2_row;
```

```
// Outputs
wire [AREA_COL*2-1:0] r1_data;
wire [AREA_COL*2-1:0] r2_data;
wire falling_update;
wire game_over;
wire [9:0] game_score;
```

```

Wrapper #(
    .AREA_ROW(AREA_ROW),
    .AREA_COL(AREA_COL),
    .ROW_ADDR_W(ROW_ADDR_W),
    .COL_ADDR_W(COL_ADDR_W),
    .SPEED_FREQ(SPEED_FREQ)
) uut (
    .clk(clk),
    .rstn(rstn),
    .pressed_left(pressed_left),
    .pressed_right(pressed_right),
    .pressed_up(pressed_up),
    .pressed_down(pressed_down),
    .pressed_speed_up_down(pressed_speed_up_down),
    .pressed_pause_or_start(pressed_pause_or_start),
    .r1_row(r1_row),
    .r1_data(r1_data),
    .r2_row(r2_row),
    .r2_data(r2_data),
    .falling_update(falling_update),
    .game_over(game_over),
    .game_score(game_score)
);

```

```

always #1 clk = ~clk;

```

```

initial begin
    // 初始化输入
    clk = 0;
    rstn = 0;
    pressed_left = 0;
    pressed_right = 0;
    pressed_up = 0;
    pressed_down = 0;
    pressed_speed_up_down = 0;
    pressed_pause_or_start = 0;
    r1_row = 0;
    r2_row = 0;

    // 复位
    #100;
    rstn = 1;
    #100;

```

```

// 模拟按键按下
#10 pressed_left = 1;
#10 pressed_left = 0;

#10 pressed_right = 1;
#10 pressed_right = 0;

#10 pressed_up = 1;
#10 pressed_up = 0;

#10 pressed_down = 1;
#10 pressed_down = 0;

#10 pressed_speed_up_down = 1;
#10 pressed_speed_up_down = 0;

#10 pressed_pause_or_start = 1;
#10 pressed_pause_or_start = 0;

// 模拟进行中
repeat (1000) begin
    #10;
end

// 检查游戏结束和得分
$display("Game Over: %b", game_over);
$display("Game Score: %d", game_score);

// 结束仿真
$stop;
end

// 下降更新信号生成
reg [25:0] falling_clk_cnt = 26'b0;

assign falling_update = (falling_clk_cnt == (SPEED_FREQ - 1)) ? 1'b1 : 1'b0;

always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        falling_clk_cnt <= 26'b0;
    end
    else if (falling_clk_cnt >= (SPEED_FREQ - 1)) begin
        falling_clk_cnt <= 26'b0;
    end
end

```

[illegible]

测试所有信号的运作状态以及下落的计数情况正常。

Segwrapper:

```
`timescale 1ns/1ns

module tb_segwrapper;

    // Parameters
    parameter SCAN_FREQ = 200;
    parameter CLK_FREQ = 50_000_000;
    parameter SCAN_CLK_CNT = 2;

    // Inputs
    reg clk = 0;
    reg rstn = 1;
    reg [9:0] game_score = 3;

    // Outputs
    wire [5:0] seg_sel;
    wire [7:0] seg_data;
```

```

// Instantiate the SegWrapper module
// Instantiate the SegWrapper module
SegWrapper #(
    .CLK_FREQ(CLK_FREQ),
    .SCAN_FREQ(SCAN_FREQ),
    .SCAN_CLK_CNT(SCAN_CLK_CNT)
) seg_wrapper_inst (
    .clk(clk),
    .rstn(rstn),
    .game_score(game_score),
    .seg_sel(seg_sel),
    .seg_data(seg_data)
);

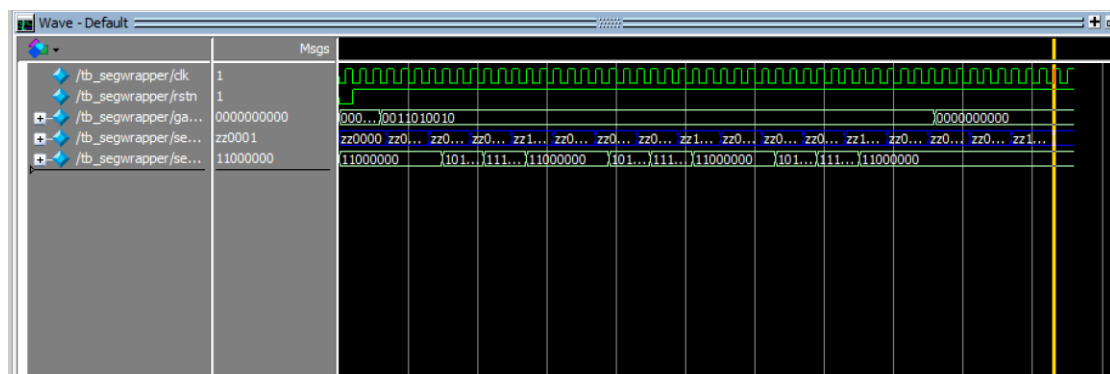
// Clock generation
always #5 clk = ~clk;

// Test sequence
initial begin
    // Apply a reset
    rstn = 0;
    #10 rstn = 1;

    // Simulate game scores
    #20 game_score = 1234;
    #400 game_score = 0000;

    // Wait for simulation to finish
    #100 $finish;
end
endmodule

```



在每个检测时期结束后的下一个上升沿时，进行对应 segment 的分数更新。

收获与思考：

1. 理解 Verilog 语言与 FPGA

通过编写代码来实现游戏的各个功能模块，我更深刻地理解了 Verilog 硬件描述语言，明白如何去解读代码，将开源的代码拆解为所用（如消抖处理）。同时，学会了如何使用 FPGA 以及如何在 FPGA 开发板上进行调试和测试，在实现功能的瞬间体会到了欣喜若狂。

2. 增强对数字（时序）电路设计的理解

在实现游戏的过程中，存在多个模块的设计和实现，比如按键消抖、方块控制、碰撞检测、行消除等。那在这其中也遇到了很多困难，尤其是对于时序的理解，以至于我一开始设计的程序虽然看起来能够运行，但在一些细节上（如方块撞墙）并不能完美呈现出合理效果，这是由于我最开始的设计思路中触发器的触发原因都是 clk 上升，但是它们实际上并不是在同一 clk 下进行，而是有先后顺序的，因此尽管其他功能能够被实现，但依旧不能成为一个好的 demo。在这个 project 的制作中，这些设计和实现让我更深入地理解了数字电路的工作原理和设计方法，特别是在时钟信号处理、同步和异步设计等方面。

3. 提高问题解决能力和逻辑思维能力

在项目中，我需要解决很多具体的问题，比如如何检测碰撞、如何判断方块的合法移动、如何处理行消除等。想要解决这些问题，我需要先抽象出我的输入和输出，了解了这些之后再牵线搭桥使二者间建立起联系，整个过程就像是身临其境地参与到一场迷宫游戏。这些问题的解决过程，极大地提高了我的问题解决能力和逻辑思维能力。